Here are **short, interview-focused notes** with **tricky questions** for each topic.

---

1. Use extends to create an inheritance relationship

Definition

extends allows a child class to inherit fields and methods from a parent class.

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Bark");
    }
}
```

Interview Points

- Child gets **non-private** members of parent.
- Java supports **single inheritance** for classes.
- Constructors are **not inherited**.

Tricky Interview Questions

Q: Can a class extend multiple classes?

> No. Java doesn't support multiple inheritance with classes.

Q: Are private members inherited?

> Yes, but they are **not directly accessible** in the child.

Q: Can a child reduce parent visibility?

> No.

Q: Can a class extend itself?

> No.

---

2. Understand IS-A vs HAS-A Relationship

IS-A (Inheritance)

```
class Animal {}

class Dog extends Animal {}
```

Dog IS-A Animal.

HAS-A (Composition)

```
class Engine {}

class Car {
    Engine engine = new Engine();
}
```

Car HAS-A Engine.

Interview Rule

- **IS-A → extends**
- **HAS-A → object/reference**

Tricky Interview Questions

Q: Which is preferred?

> HAS-A (Composition), because it's more flexible.

Q: Can inheritance replace composition?

> Not always. Use inheritance only for true IS-A relationships.

Q: "Student has a Name." Which relationship?

> HAS-A.

---

3. Use super to call parent constructor or parent method

Parent Constructor

```
class Animal {
    Animal() {
        System.out.println("Animal");
    }
}

class Dog extends Animal {
    Dog() {
        super();
    }
}
```

Parent Method

```
class Animal {
    void sound() {
        System.out.println("Animal");
    }
}
```

```
class Dog extends Animal {
    void sound() {
        super.sound();
        System.out.println("Dog");
    }
}
```

Interview Points

- super() calls parent constructor.
- super.method() calls parent method.
- Must be **first statement** in constructor.

Tricky Interview Questions

Q: What happens if super() isn't written?

> Java inserts it automatically (if parent has a no-arg constructor).

Q: Can super() and this() appear together?

> No.

Q: Can super() appear outside constructor?

> No.

Q: Can you access private parent fields using super?

> No.

---

4. Override methods using @Override

Example

```
class Animal {
    void sound() {
        System.out.println("Animal");
```

```
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

Rules

- Same method name
- Same parameters
- Same (or covariant) return type
- Child cannot reduce access level

Tricky Interview Questions

Q: Can static methods be overridden?

> No. They are hidden.

Q: Can private methods be overridden?

> No.

Q: Can final methods be overridden?

> No.

Q: Why use @Override?

> Compiler catches mistakes if the method doesn't actually override.

Q: Is overriding Runtime or Compile-time polymorphism?

> Runtime.

---

5. Overload methods using different parameter lists

Example

```
class MathUtil {

    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Rules

Must differ by:

- Number of parameters
- Type of parameters
- Order of parameter types

Return type alone **cannot** overload.

Tricky Interview Questions

Q: Can methods be overloaded only by return type?

> No.

```
int test()
double test()   // □ Compile Error
```

Q: Is overloading compile-time or runtime polymorphism?

> Compile-time.

Q: Can constructors be overloaded?

> Yes.

Q: Can static methods be overloaded?

> Yes.

🔥 Very Common Interview Rapid Fire

- **Difference between Overloading & Overriding?**
  - Overloading → Same class, different parameters, compile-time.
  - Overriding → Parent-child, same signature, runtime.
- **Why doesn't Java support multiple inheritance with classes?**
  - To avoid the **Diamond Problem** and ambiguity.
- **Which is more flexible: Inheritance or Composition?**
  - Composition (HAS-A).
- **Can a child constructor run before the parent constructor?**
  - No. Parent constructor always executes first.
- **Can abstract methods be overridden?**
  - Yes, they must be implemented by the first concrete subclass.
- **Can an overridden method throw a broader checked exception?**
  - No. It can throw the same, a narrower checked exception, or any unchecked exception.

These are the core concepts and interview traps that are frequently asked for these five topics in Java backend and internship interviews.